

A Simulation Framework for the Multi-Period Vehicle Routing Problem with Profits in Smart Waste Collection

No Author Given

¹ INESC-ID, Lisboa, Portugal afonso.fernandes@inesc-id.pt

² Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal
afonso.fernandes@tecnico.ulisboa.pt

Abstract. Urban waste collection is a critical municipal service with profound implications for sanitation and public health. This service heavily relies on vehicle routing and has traditionally been modeled as a static Capacitated Vehicle Routing Problem (CVRP), where vehicles perform an exhaustive collection of all bins. The introduction of smart waste bin monitoring technology has enabled dynamic vehicle routing, allowing vehicles to selectively visit only the subset of bins that require immediate service or are profitable to collect, a task which can be modeled as a Vehicle Routing Problem with Profits (VRPP). Furthermore, this task must be performed across an extended planning horizon, where routes executed in a given period impact the optimal routes for subsequent periods, forming a Multi-Period VRPP (MPVRPP). Because of the large scale and temporal dependencies, transferring existing CVRP solvers to this selective routing problem presents significant structural difficulties. Moreover, progress is currently hindered by a lack of standardized multi-period evaluation environments. To address these limitations, this paper introduces WSmartRoute+, a Python simulation framework designed specifically for the stochastic MPVRPP. Using this environment, a unified methodology is developed to adapt state-of-the-art CVRP algorithms - spanning exact methods, meta-heuristics such as Hybrid Genetic Search (HGS) and Adaptive Large Neighborhood Search (ALNS), and hyper-heuristics - to dynamic waste collection. The adaptations introduce novel profit-aware repair operators, and dynamic dispatch heuristics. The framework composes any mandatory-selection strategy, route constructor and route improver as interchangeable stages, and integrates Neural Combinatorial Optimization (NCO) constructors on the same interface as the classical ones, so that learning-based and classical solvers can be evaluated under identical demand realisations; the benchmark reported here exercises eight classical constructors across a complete factorial design, and the learned constructors are left to future work. The experimental results establish a unified baseline for the MPVRPP and show that the choice of which bins to collect dominates the choice of how to route them, with efficiency varying by 94% across selection strategies against 24% across constructors - demonstrating WSmartRoute+ as a testbed for future research in dynamic, multi-period waste collection routing.

1 Introduction

Urban waste collection is a core public service with significant economic, environmental, and social implications. The operational planning of waste collection routes directly affects fuel consumption, labor costs, greenhouse gas emissions, service reliability, and citizen satisfaction. From a methodological standpoint, waste collection routing represents a particularly challenging class of vehicle routing problems, characterized by spatially distributed service points, capacity constraints, heterogeneous waste generation rates, and strict operational requirements.

Over the past decades, Operations Research (OR) has produced a wide range of optimization-based models and solution methods for waste collection routing, including exact formulations, decomposition approaches, and advanced heuristics [2,3,11,13]. While these approaches are capable of modeling realistic constraints and multiple objectives, their evaluation is typically conducted either on limited real instances or on static snapshots of the system. As a result, important operational aspects—such as daily variability in waste accumulation, interdependence between routing decisions over time, and the long-term effects of routing policies—are often neglected.

In parallel, recent advances in Deep Learning (DL) have given rise to Neural Combinatorial Optimization (NCO) methods, which aim to learn routing policies directly from data using deep neural networks trained via reinforcement learning [18]. These approaches have shown promising results on classical routing benchmarks, particularly when evaluated on synthetic and static problem instances. However, their applicability to real-world waste collection operations remains largely unexplored, due in part to the lack of realistic, dynamic, and reproducible evaluation environments.

A critical limitation of the current literature is the absence of simulation frameworks that faithfully capture the operational dynamics of waste collection systems. Real-world operations involve continuous waste accumulation, sensor uncertainty, service-level requirements, and feedback effects whereby routing decisions made on one day influence the state of the system on subsequent days. Without a simulation layer that explicitly models these dynamics, it is difficult to assess the true performance, robustness, and scalability of both OR-based solvers and learning-based routing policies.

To address this gap, this paper introduces a Waste Collection Routing Simulator designed to serve as a unified experimental platform for the development, evaluation, and comparison of routing strategies. The simulator integrates a dynamic system state that evolves over time, daily route execution, container-level waste accumulation processes, and multi-objective performance metrics reflecting real operational trade-offs. By decoupling the simulation of the operational environment from the optimization or learning components, the framework enables a fair and reproducible comparison between expert-crafted OR solvers and data-driven NCO models under identical conditions.

Within this simulator, routing decisions are not evaluated in isolation but as part of a sequential decision-making process, where each day’s routes influence

future system states, service feasibility, and performance outcomes. This perspective shifts the focus from solving single-shot routing problems to evaluating routing policies over extended planning horizons, which is more aligned with real-world waste management practice.

The proposed simulator is validated using real-world data from a waste management company operating across multiple municipalities in Portugal, involving instances with several hundreds of containers. It provides a realistic testbed for studying the strengths and limitations of both optimization-based and learning-based routing approaches, and supports systematic experimentation on scalability, robustness to uncertainty, and multi-objective trade-offs in waste collection operations.

This paper makes the following contributions:

- We introduce a dynamic, multi-objective waste collection routing problem in which routing decisions influence the evolution of future problem instances through container-level waste accumulation and service constraints. This formulation naturally captures the sequential nature of real-world waste collection operations.
- We propose a modular Waste Collection Routing Simulator that enables the evaluation of routing policies under both blind and smart collection settings. The simulator supports stochastic waste generation processes, sensor uncertainty, and long-term policy evaluation over extended planning horizons.
- We develop and adapt Neural Combinatorial Optimization architectures for dynamic waste collection, including a lightweight attention-based model inspired by recommendation systems, and benchmark them against state-of-the-art expert-crafted OR solvers [3,11,13].
- We conduct an extensive computational study using real-world data from a Portuguese waste management company, considering networks of 100, 170, and up to 350 containers. The experiments analyze scalability, robustness to uncertainty, and trade-offs between operational cost, service level, and environmental impact.
- We systematically investigate the impact of different waste generation models, including Gamma distributions with varying parameters and empirical distributions derived from historical data, providing insights into how demand uncertainty affects the relative performance of static and dynamic routing policies.

2 Problem Definition

We model the Multi-Period Vehicle Routing Problem with Profits (MPVRPP) as a profit maximizing, multi-period constrained combinatorial optimization problem over a horizon of D days.

2.1 Problem State and Dynamics

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a fully connected graph where $\mathcal{V} = \{v_0, v_1, \dots, v_V\}$, with v_0 representing the depot and $\mathcal{V} \setminus \{v_0\}$ denoting the set of waste bins located at spatial

coordinates $\vec{l}_i \in \mathbb{R}^2$. The state of the environment at day $d \in \{1, \dots, D\}$ is defined by the fill levels of the bins, denoted by the vector $\vec{w}_d = [w_{1,d}, \dots, w_{V,d}]^\top \in [0, \tau]^V$, where τ is the maximum bin capacity.

The waste $w_{i,d}$ accumulates stochastically according to a daily waste fill value sampled from a bin-specific distribution $\delta_{i,d} \sim P_i$. If a bin v_i is visited on day d , its waste is collected and the level is reset; otherwise, waste continues to accumulate up to the bins' maximum capacity τ :

$$w_{i,d+1} = \begin{cases} \delta_{i,d+1} & \text{if } v_i \in \mathcal{A}_d \\ \min(w_{i,d} + \delta_{i,d+1}, \tau) & \text{if } v_i \notin \mathcal{A}_d \end{cases} \quad (1)$$

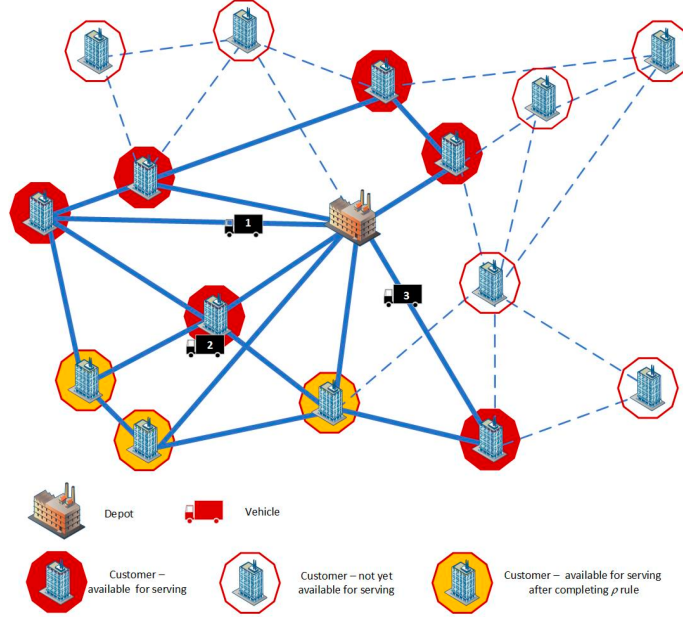


Fig. 1. Example of a single-day VRPP where multiple vehicles each serve a different subset of the selected bins. Figure adapted from [8].

2.2 Optimization Objective

At each day d , the agent selects a (possibly empty) set $\mathcal{A}_d$ of bins to be visited distributed by K routes, with each route starting and ending at the depot v_0 :

$$\mathcal{A}_{d,k} = (a_{0,k}, a_{1,k}, a_{2,k}, \dots, v_0). \quad (2)$$

Here $a_{t,k}$ represents the action of visiting a certain bin on route k , with $a_{0,k} \equiv v_0$. The maximum number of routes can be limited by the resources

available (vehicles). The constrained optimization problem for a single day can be formulated as finding the route(s) that:

$$\begin{aligned}
& \text{maximize} \quad \mathcal{P} = r_w \cdot \sum_{v_i \in \mathcal{A}_{d,\cdot}} w_{i,d} - c_{km} \cdot \sum_{k=1}^K \sum_{a_{t,k} \in \mathcal{A}_{d,k}} \text{dist}(a_{t,k}, a_{t+1,k}) \\
& \text{subject to} \quad a_{t,k} \neq a_{t',k'} \quad \forall t, k \neq t', k' \\
& \quad \quad \quad \sum_{v_i \in \mathcal{A}_{d,k}} w_{i,d} \leq Q_k
\end{aligned} \tag{3}$$

where r_w is a scalar value for the revenue obtained from each waste kilogram collected, c_{km} is a scalar value corresponding to the cost it takes for a vehicle to travel one kilometer, a_t represents the action of going to a certain bin, and Q_k is the vehicle maximum waste capacity constraint.

The Vehicle Routing Problem with Profits (VRPP) can be seen as a variation of the Vehicle Routing Problem (VRP), which is in itself an generalization of the TSP where each vertex has a demand and routes must be constructed such that they both start and end at the depot [7,18]. Unlike the base VRP, the VRPP's objective function is the profit, which considers the demand collected in addition to the route travel cost, thus visiting only the subset of nodes that is profitable, instead of always visiting every available node [1]. Additionally, for the Multi-Period variant of the VRPP (MPVRPP), the vehicles must perform multiple collection routes during a given time range, where the current day's optimal route is dependent upon the choices taken at previous days [19].

Since decisions made on day d affect the state on day $d+1$, we must optimize over the full horizon. The global objective is to find a policy π that generates a sequence of routes $(\mathcal{A}_1, \dots, \mathcal{A}_D)$ to maximize the expected cumulative profit:

$$\max_{\pi} \mathbb{E} \left[\sum_{d=1}^D \mathcal{P}(\mathcal{A}_{d,\cdot}, \mathbf{w}_d) \right], \tag{4}$$

where $\mathcal{P}$ is the daily route(s) profit function defined in (3).

3 Related Work

3.1 Neural Combinatorial Optimization

The framework registers NCO constructors on the same interface as the classical ones, so the taxonomy below defines the design space those constructors are drawn from. They are not part of the benchmark reported in Sect. 5, which exercises the eight classical constructors; the purpose of this subsection is to establish what a learned constructor would have to be, and where in the pipeline it would sit.

Following a slightly modified version of the taxonomy introduced by [18], which also considers the categories defined by [15], NCO solvers can be classified

into five different types based on how the DNN model generates the solution: 1) autoregressive constructive solvers [7,9,17], which start from an empty set and build a valid solution by sequentially selecting unvisited vertices; 2) non-autoregressive constructive solvers [5,6], where the DNN model learns to output the entire solution (usually consisting of probabilities over the vertices or edges of the graph, that are then used by a decoding algorithm to build a valid route) in a single step; 3) improvement heuristic solvers [12], which use a Markov Decision Process (MDP) to iteratively refine an existing complete solution; 4) predict-once solvers [15], where the model predicts information before starting a solution search with a OR solver or a search algorithm (like beam search); and 5) predict-many solvers [10], a similar approach to the previous category, with the difference that the model performs additional predictions to further restrict the solution search space, and thus decrease time complexity. The NCO solvers from different categories provide various trade-offs between performance, solving time, and ability to scale to larger problem instances [4,15,18].

While all of these algorithms share the same purpose, to produce a near-optimal solution to a given CO problem instance, in practice they learn to perform different specific tasks using a plethora of methods from multiple ML paradigms, including training to mimic OR solvers via SL [5,15,17], approximating the optimal policy by minimizing a cost function via policy gradient RL [7,9,12], and using Unsupervised Learning (UL) to make a model define a probability distribution over a graph’s vertices that is used at inference time to build a feasible route through the method of conditional expectation [6].

4 Methodology

A routing policy in WSmartRoute+ is not a single algorithm but a composition of three independent stages, each drawn from its own plugin registry (Fig. 2). The first is *mandatory selection*: given the sensed fill level of every bin, decide which bins *must* be serviced today. The second is *route construction*: build a route over those mandatory bins, free to add any further bin whose waste repays the detour needed to reach it. The third, optional, stage is *route improvement*: take the constructed route and refine it.

Separating the three matters because they answer different questions. Selection is a multi-period question — collecting a bin today forgoes the larger, cheaper-per-kilogram load it would offer tomorrow — while construction is a single-period profit-maximising routing question, and improvement is a pure geometry question. Composing them freely means an experiment can vary one stage while holding the others fixed, which is what makes the marginal effect of each stage measurable at all. It is also what turns a benchmark of eight algorithms into a benchmark of a policy *space*: the codebase registers 32 selection strategies, more than 90 route constructors across seven families, and 33 route improvers, all interchangeable by configuration rather than by code.

This work benchmarks a deliberately small, representative subset of that space: three selection strategies (in five threshold variants), eight route constructors

spanning exact methods, meta-heuristics and hyper-heuristics, and two route improvers.

Fig. 2. The policy configuration space. A policy is one choice from each of the three stages; the experiments in Sect. 5 cross the highlighted subset exhaustively.



4.1 Mandatory Selection Strategies

All three strategies decide from *sensed* fill levels, never from the true ones, and all three must estimate whatever they need about accumulation from the history the simulation has produced so far. Neither the generating distribution nor its parameters are ever revealed to them. The three were chosen to span the conceptual space rather than to be exhaustive: one reactive, one statistical, one simulation-based.

Last-Minute (LM). The simplest possible rule, and the reactive baseline: a bin becomes mandatory when its sensed fill level crosses a fixed critical-fill threshold, judged in complete isolation from every other bin and from any estimate of how fast it is filling. Formally, bin i is mandatory on day t iff $\hat{f}_{i,t} \geq \tau$, for sensed fill $\hat{f}_{i,t}$ and threshold τ . We test $\tau = 0.7$ (CF70) and $\tau = 0.9$ (CF90). The threshold is the only knob, and it trades overflow risk against visit frequency directly, with no model connecting the two.

Service-Level (SL). A statistical rule that projects forward rather than reacting. For each bin it maintains online estimates of the mean daily accumulation $\hat{\mu}_i$ and its standard deviation $\hat{\sigma}_i$ from the days observed so far, and flags bin i as mandatory when a conservative projection of its fill h days ahead already exceeds capacity C_i :

$$\hat{f}_{i,t} \cdot C_i + h\hat{\mu}_i + z\hat{\sigma}_i\sqrt{h} \geq C_i, \quad (5)$$

where z is a fixed safety coefficient. We test projection horizons $h = 1$ (SL1) and $h = 2$ (SL2), holding z constant, so the two variants differ in how far ahead they look and not in how conservative they are at a given distance.

Two caveats apply to reading this as a probabilistic guarantee. The safety margin approximates a Gaussian quantile, whereas the waste actually generated in our scenarios is right-skewed; and $\hat{\mu}_i$, $\hat{\sigma}_i$ are cold-start estimates that are least reliable early in a horizon, exactly when the strategy has had least opportunity to recover from a mistake.

Look-Ahead (LA). A simulation-based rule that assumes no distributional form at all. Rather than projecting analytically, it rolls the bin's accumulation forward day by day to find the last day on which the bin could still be collected without

overflowing, and makes it mandatory only on that day — deferring collection to the latest safe moment so that the collected load is as large as possible. It has no threshold variant, which is why it contributes fewer configurations than LM or SL to the experimental grid. It is also the most expensive of the three per decision, since it simulates forward instead of evaluating a closed form.

4.2 Route Constructors

Every constructor below solves the same single-period problem: visit all mandatory bins, optionally visit any subset of the remaining bins, respect vehicle capacity, and maximise collected waste net of travel cost. The adaptation that matters is uniform across the families — classical CVRP methods assume every node must be visited, and each constructor needs some mechanism to decide that a bin is *not* worth the detour.

Exact Methods

Smart Waste Collection — Two-Commodity Flow (SWC-TCF). A direct implementation of the waste-collection formulation of Ramos et al. [14], written as a single monolithic mixed-integer program and handed to a general-purpose solver. Its core device is to track two commodities on every arc simultaneously — waste collected so far, and residual capacity — and to require that they sum to the vehicle capacity on every used arc. That single constraint family enforces capacity and eliminates subtours at once, where a conventional formulation needs two separate and awkward families.

Unlike the other constructors, SWC-TCF needs no VRPP retrofit: the source formulation already treats “is bin i visited” as a first-class binary, with the objective trading collection revenue against travel cost and a per-vehicle usage penalty directly. Bins flagged mandatory, or already critically full, are forced into the solution regardless.

Its exposure is scale. The model grows roughly quadratically in the number of candidate bins and there is no decomposition to keep the working problem small, so it is the constructor most vulnerable to instance-size blow-up — a vulnerability that our largest scenario realises concretely (Sect. 5.3).

Branch-Price-and-Cut (BPC). The decomposition-based exact method. The master problem is a route-selection linear program in which each column is one complete candidate route; routes are generated on demand by a pricing subproblem rather than enumerated, since the number of possible routes is astronomically large for any non-trivial instance. Pricing solves a resource-constrained shortest-path problem whose arc costs incorporate the master problem’s current duals, so a route is only generated if it could improve the incumbent. Because an exactly elementary path search is itself intractable at this scale, the pricer works with near-elementary paths and tightens that relaxation adaptively when generated routes begin revisiting bins too often.

Three mechanisms together give BPC its profitable-subset behaviour. Coverage constraints are *at most once* for optional bins and *exactly once* for mandatory ones; the pricing subproblem maximises profit rather than minimising distance, so route extension stops naturally once a detour costs more than the waste it collects; and branching resolves the visit/no-visit decision for a bin *before* it branches on arcs, settling the question a classical always-cover formulation never has to ask. Beyond the base formulation, lifted cover inequalities on saturated arcs are separated — chosen to match the reference methodology rather than the now more common rounded capacity cuts — together with VRPP-specific minimum-cut, triangle-clique and node-profit families.

Both exact methods run under a per-run time budget in these experiments. Neither is run to certified optimality on the larger instances, and both have a documented non-exact fallback on timeout: BPC returns a greedily constructed plan, SWC-TCF its best incumbent. Their results should be read as “best solution found within budget”, not as proven optima.

Meta-Heuristics

Adaptive Large Neighborhood Search (ALNS). A trajectory method that maintains one incumbent and repeatedly destroys and repairs part of it. Its adaptivity is in operator choice: it holds a portfolio of destroy operators (random removal, worst removal, and Shaw removal, which removes a set of mutually related bins so that repair has the freedom to place them jointly) and repair operators (greedy insertion and regret- k insertion, optionally with randomised noise on insertion costs), and reweights them by recent success. Operator i is drawn by roulette wheel from weights updated every φ iterations:

$$w_{i,t+1} = \lambda w_{i,t} + (1 - \lambda) \frac{\pi_i}{\theta_i}, \quad (6)$$

where $\lambda \in [0, 1]$ is a decay factor, π_i the score accumulated by operator i during the segment, and θ_i its usage count. Candidates are accepted by a simulated-annealing rule, so the trajectory can escape local optima. The profitable-subset decision falls out of repair: a removed bin for which no insertion position yields positive marginal profit is simply left out.

Hybrid Genetic Search (HGS). A memetic algorithm following Vidal [16], and the population counterpart to ALNS. Its central design choice is representational: individuals are encoded as a single giant tour over all bins with no route boundaries, and are decoded into actual routes only when evaluated. Decoding is a dynamic program that finds the optimal split of the giant tour into capacity-feasible segments — and, critically for VRPP, it is also where bins are *skipped*, by allowing the split to jump over any bin whose inclusion would reduce net profit. Skipped bins remain in the genotype, so their relative position survives into later generations in case a subsequent mutation makes them worth collecting again.

Two sub-populations are maintained, one feasible and one infeasible-but-penalised, so that promising infeasible individuals survive long enough to be

repaired. Each generation selects two parents by a tournament biased toward both fitness and diversity, recombines them, and educates the offspring by local search over relocate, 2-opt and the algorithm’s signature SWAP* move, which searches for each exchanged node’s genuinely best new position rather than only the partner’s vacated slot, pruned to angularly nearby sectors to stay tractable. Fitness combines cost $C(S)$ with a diversity contribution $D(S)$:

$$F(S) = C(S) + \left(1 - \frac{|P_{feas}|}{|P|}\right) \cdot D(S). \quad (7)$$

Simulated Annealing Neighborhood Search (SANS). An implementation of the hybrid published by Jorge et al. [3] specifically for smart waste collection. It couples the standard annealing acceptance rule — accept improving moves always, accept a worsening move of magnitude Δ with probability $\exp(-\Delta/T)$ under a temperature T that cools over the run — with a broad, purpose-built move library: single-bin relocation and swapping within and between routes, batched relocation and swapping of both random and contiguous bin groups, segment reversal, and route-segment crossover. Moves are sampled uniformly rather than adaptively, which is the substantive difference from ALNS: SANS steers entirely through acceptance, never down-weighting an unproductive operator.

The profitable-subset decision is folded into the neighbourhood itself. Bins can be moved into a removed pool and reconsidered for reinsertion later, so inclusion is one more move type rather than a separate phase. A refinement stage follows the annealing loop: an uncrossing pass, further annealing, deterministic local search applied twice — once in forward order and once with routes and positions reversed, which escapes the local optima that an order-dependent sweep creates — and a final pass over the removed pool.

Pheromone-Guided Cooperative Large Neighborhood Search (PG-CLNS). An original design combining population-level construction with per-individual refinement. Construction is ant-colony-based, with pheromone deposited on *edges* of the bin graph, so that road segments appearing in good solutions are more likely to be chosen again; only the global best solution deposits into the shared trail each round, keeping the signal focused, while a temporary within-round penalty discourages ants from reusing edges another ant has just taken. Refinement is large neighbourhood search, run independently on every member of the population with its own adaptive destroy/repair weighting.

The two levels adapt on different objects and at different scopes: pheromone is edge-based and population-wide, operator weights are move-based and per individual. Each round constructs a population under the current trail, refines every member, updates the trail from the round’s best, and regenerates the weakest fraction of the population under the updated field. Profitability is screened at insertion in both phases, as in ALNS.

Because every member is refined every round, total work scales with population size times refinement depth times rounds; on non-trivial instances the algorithm is typically stopped by its time budget rather than by convergence.

Particle Swarm Optimization Memetic Algorithm (PSOMA). Particle swarm optimisation is a continuous method, and applying it to a sequencing problem requires an encoding. PSOMA uses ranked-order values: each particle holds a real vector with one coordinate per bin, and the visiting order is simply the rank order of those coordinates. The numbers themselves are meaningless; only their relative order is read, so the standard continuous velocity update — inertia, plus attraction to the particle’s own best and to the swarm’s best — runs unmodified underneath, with the discrete tour emerging from a sort. The resulting giant tour is split into routes by the same profit-aware dynamic program HGS uses, which is likewise where PSOMA decides which bins to skip.

The memetic component refines the swarm’s incumbent once per iteration by a simulated-annealing local search over swap, relocate and segment-reversal moves, adaptively favouring whichever move type has recently been productive. A secondary heuristic biases the local search’s cost estimates so that pairs of profitable bins appear closer together, steering cheap moves toward retaining valuable bins without paying for a full route decode.

Hyper-Heuristics

Ant Colony Optimization Hyper-Heuristic (ACO-HH). The one true hyper-heuristic in the benchmark, and worth contrasting directly with PG-CLNS, which uses the same metaphor one level lower. Where classical ACO lays pheromone on edges of the *bin* graph to decide which bin to visit next, ACO-HH lays pheromone on transitions in a graph whose nodes are the available low-level heuristics, and so decides which *move to apply* next. It searches the space of operator sequences and delegates all solution modification to the operator it selects.

An ant walks this operator graph, applying each move it steps onto to the working solution. The next operator is drawn with probability

$$p_{sh} = \frac{[\tau_{sh}]^\alpha [\eta_{sh}]^\beta}{\sum_{k \in \mathcal{H}} [\tau_{sk}]^\alpha [\eta_{sk}]^\beta}, \quad (8)$$

where τ_{sh} is the pheromone on the transition from state s to heuristic h , η_{sh} is heuristic information about h ’s likely efficacy, and α, β weight the two. If a completed walk improved the solution, pheromone is deposited along every transition it used, so operator *sequences* that work well together are reinforced rather than merely individually good operators; a faster-adapting efficiency signal that accounts for each operator’s runtime modulates selection on a shorter timescale, and all pheromone evaporates so that stale sequences fade.

Two separate portfolios are involved: construction heuristics (greedy, nearest- and farthest-insertion, regret-based, each with a profit-aware variant) used once to build the initial solution, and the modification operators the ant colony actually searches over. Inclusion decisions are handled by the profit-aware variants throughout, supplemented by a strategic-oscillation mechanism that periodically relaxes the capacity penalty when the search stagnates, letting it revisit inclusion decisions it would otherwise be trapped away from.

4.3 Route Improvers

Both improvers take a constructed route plan and refine it, and neither can change which bins were selected — a property that the experiments confirm directly (Sect. 5.4).

Classical Local Search (CLS). Variable-neighbourhood descent over a fixed toolbox ordered from cheapest to most expensive to evaluate: relocate a bin, swap two bins, reverse a segment (2-opt), relocate a chain of two or three consecutive bins (Or-opt), exchange route tails between vehicles, exchange bins between routes with re-optimised placement, and finally full three- and four-bin reordering. Whenever any move succeeds the search restarts from the cheapest neighbourhood rather than continuing, and it terminates at a local optimum with respect to every move in the toolbox.

One property matters for interpreting the results: every CLS move is scored purely by its effect on travel distance. Capacity is enforced as a feasibility check, but collected revenue never enters the accept/reject decision, so CLS optimises distance and not profit. It can, however, move a bin between vehicles.

Fast-TSP. A narrower operation: holding each route’s bin assignment fixed, find the best visiting order within it. Routes of up to roughly twenty stops are solved to optimality by dynamic programming; longer ones fall back to a randomised local search within a small fixed budget. Fast-TSP cannot move a bin between vehicles, drop a bin, or account for revenue — it solves each route’s ordering globally where CLS improves the plan incrementally, and that difference is precisely what the paired comparison in Sect. 5.4 isolates.

4.4 Simulation Protocol

Each simulated day executes a fixed cycle: bins accumulate waste; the selection strategy reads sensed fill levels and flags mandatory bins; the constructor builds a route and the improver refines it; the route is executed and the collected waste recorded; the day is logged and the state carried forward.

Four properties of this loop bear on how the results should be read.

Comparisons are paired at the level of the demand realisation. The simulator re-seeds its waste generation per policy-and-day combination rather than letting each run consume randomness in its own order, so every algorithm in a given scenario faces the literal identical sequence of daily fill levels, bin by bin and day by day. Differences between algorithms in the same scenario are therefore differences in decision-making, not in the draws they happened to face. This also underwrites the integrity check of Sect. 5.3: within a scenario the total waste available to collect is fixed, so collected tonnage is directly comparable across constructors.

Selection acts on noisy observations. Strategies see a sensed fill level perturbed by random error; overflow and loss accounting always uses the true level. A

strategy can therefore make a wrong call purely from what its sensors reported, as in a real deployment.

Overflow is a flag, loss is a quantity. A bin’s true level is tracked continuously; waste beyond capacity is recorded as lost and the displayed level capped. An overflowing bin that stays unvisited does not accrue a special debt — it simply keeps losing whatever it accumulates, every subsequent day, until collected. Overflow counts and kilograms lost are therefore related but distinct measures of service failure.

Every scenario uses a single vehicle and a single depot. This is a deliberate scope restriction and a real limitation: multi-vehicle and multi-depot dispatch is not exercised anywhere in these experiments, and the capacity constraint is delegated to each constructor’s own internal mechanism rather than policed centrally by the simulator.

5 Experimental Evaluation

5.1 Scenarios and Data

Experiments are run on two Portuguese municipal networks, each with real bin and depot coordinates and a real road-distance matrix:

- **Rio Maior**, at $N = 100$ and $N = 170$ plastic-waste bins, with road distances from the Google Maps distance matrix;
- **Figueira da Foz**, at $N = 350$ bins, with road distances from OpenStreetMap.

Two waste-generation processes drive each network, chosen to contrast a parametric model against real observations:

- **Gamma-3**, drawing daily accumulation from fitted Gamma distributions ($\alpha \in \{1, 3\}$, $\beta \in \{8, 6\}$; means 8, 6, 24 and 18 kg/day with variances 64, 36, 192 and 108) with different presets assigned to different groups of bins, so accumulation behaviour varies across the network and not merely in scale;
- **Empirical**, resampling directly from historical daily-fill records collected by the WSmart+ Bin Analysis module between 2021-01-12 and 2023-10-23 — a bootstrap with no parametric assumption, replaying observed patterns.

Neither process reveals its parameters to any selection strategy; all accumulation statistics a strategy uses are estimated online from the days it has observed (Sect. 4.4). Two planning horizons are used, 30 and 90 days. The map of the Figueira da Foz network appears in Fig. 3.

5.2 Design of Experiments

The 30-day design is a complete factorial crossing of every policy stage with every scenario:

$$\underbrace{8}_{\text{constructors}} \times \underbrace{5}_{\text{selection variants}} \times \underbrace{2}_{\text{improvers}} \times \underbrace{2}_{\text{distributions}} \times \underbrace{3}_{\text{network sizes}}$$

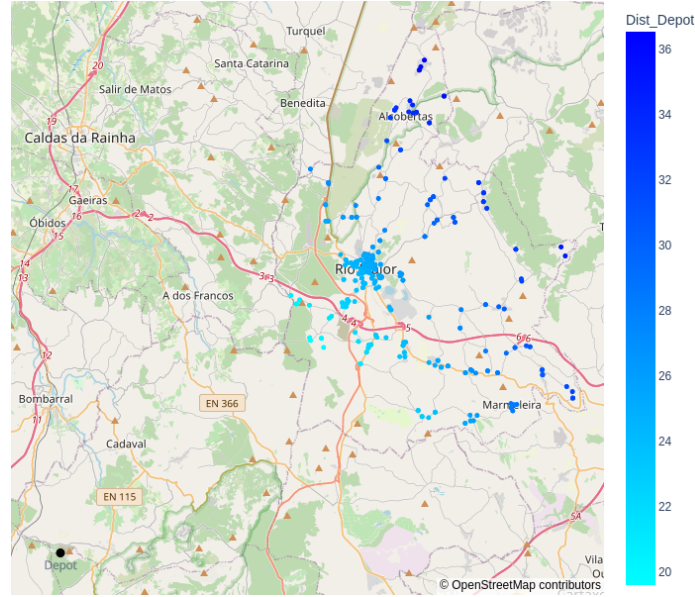


Fig. 3. Depot and bin coordinates for the larger of the two networks.

yielding 480 simulation runs, balanced so that each constructor appears in every scenario an equal number of times. Because the demand realisation is fixed per scenario (Sect. 4.4), the marginal mean of any one stage is a genuine like-for-like comparison rather than a comparison confounded by which scenarios that stage happened to be run in.

The 90-day design is *not* a factorial replication of this, and the distinction is important enough to state before any 90-day number is read. Only policy configurations that lay on the 30-day Pareto front were carried forward, producing 174 runs. Each constructor’s 90-day rows are therefore the scenarios in which it had already performed well at 30 days, and a cross-creator comparison at 90 days would compare each constructor against a different, self-favouring subset. We report no such comparison. The horizon analysis in Sect. 5.4 instead pairs each configuration against *itself* across the two horizons.

We report five measures. *Efficiency* (kg/km) is collected waste per kilometre driven and is the primary operational objective. *Overflow events* counts bin-days on which a bin was at capacity, and *kilograms lost* the waste discarded as a result; the two are related but distinct, since a single bin left uncollected keeps losing waste every subsequent day. *Distance* and *runtime* measure operational and computational cost. Means are reported alongside medians throughout, because at this sample size a single extreme scenario can move a mean substantially — as Sect. 5.3 demonstrates.

5.3 Data Integrity

Four runs are degenerate rather than merely poor, and all four are SWC-TCF on Figueira da Foz under Gamma-3 (Table 5). Because every constructor in a scenario collects from an identical waste realisation, collected tonnage is comparable by construction; these runs collected between 30% and 86% less than their scenario median, having failed to complete rather than having routed badly. Their very large overflow counts — up to 23,886 events for the 90-day case, which reached day 13 of 90 — measure that truncation, not policy behaviour. This is the concrete realisation of the scaling exposure noted for the monolithic MILP formulation: it is the largest network under the heavier of the two generating processes.

The detection rule is a tonnage shortfall against the scenario median, and its threshold is not tuned. Outside these four runs the shortfall distribution has median 0.00 and does not exceed 0.071; the four sit at 0.30, 0.47, 0.47 and 0.86. The gap identifies them unambiguously. We note that the count of collection days is *not* a truncation signal, despite appearances — several of the strongest policies collect on only 15 of 30 days precisely because they bundle collections well, so tonnage is the honest measure.

Excluding these runs alone would have introduced a subtler bias of the same family as the 90-day sampling problem: since all four are SWC-TCF’s, removing only them would average SWC-TCF over the scenarios where it did not fail while averaging its rivals over those scenarios too. We therefore drop the affected scenario cells for *all* constructors, at a cost of 23 further runs, leaving 456 runs at 30 days with every constructor contributing exactly 57.

The same reasoning applies to every other marginal comparison, in a form that is easy to miss. Dropping a scenario cell removes all constructors together, but it removes only *one* selection variant and *one* improver, since those identify the cell. Marginal tables over those stages are therefore additionally restricted to the slices in which every level of the compared stage actually ran. All tables and figures in this section are generated from the simulation output by a single tool that applies these rules centrally, so no reported number is transcribed by hand and the paper cannot drift from the data.

5.4 Results

Route Constructors Table 1 reports per-constructor performance, and Fig. 4 places the constructors in the efficiency/service plane.

BPC attains the highest mean efficiency (6.65 kg/km) on the shortest mean routes (4,018 km), which is the expected outcome for the method with the strongest optimality machinery — though, as noted, it runs to a time budget and is not certified optimal at these sizes. ALNS achieves the fewest mean overflows (5.9) and PG-CLNS is the strongest all-round compromise, second on efficiency and second on overflows. ACO-HH is the most interesting entry: third on efficiency, within 5% of BPC, at 589s mean runtime — roughly a quarter of HGS’s and PG-CLNS’s, and the fastest of the eight. The hyper-heuristic’s extra level of abstraction pays for itself.

The medians qualify the overflow column sharply, and this is the result we would most caution against over-reading. Seven of the eight constructors have a *median* overflow count of exactly 4.0; the spread in the means, from 5.9 for ALNS to 19.9 for HGS, is therefore almost entirely tail behaviour rather than typical-case behaviour. The statement “HGS overflows more than ALNS” is true of the mean and false of the median. What separates the constructors on this measure is not how they behave on an ordinary day but how badly they fail on an unusual one.

Pareto-front membership across the six scenario–distribution combinations, recomputed after the integrity filter, gives PG-CLNS 5 of 6, PSOMA and HGS 3 each, BPC 2, and ACO-HH and ALNS 1 each. PG-CLNS is thus the most consistently non-dominated constructor even though it never leads a single column outright, which is the characteristic signature of a good compromise method.

Runtime separates the constructors more cleanly than quality does (Fig. 5). At $N = 350$ the spread is 4.5-fold, from ACO-HH at 1,219s to SANS at 5,451s. Notably, BPC at 2,430s is *not* the most expensive method: PG-CLNS, HGS and SANS all cost more wall-clock time per run than the exact branch-price-and-cut solver at this scale. The common intuition that the exact method is the expensive one does not survive contact with these instances.

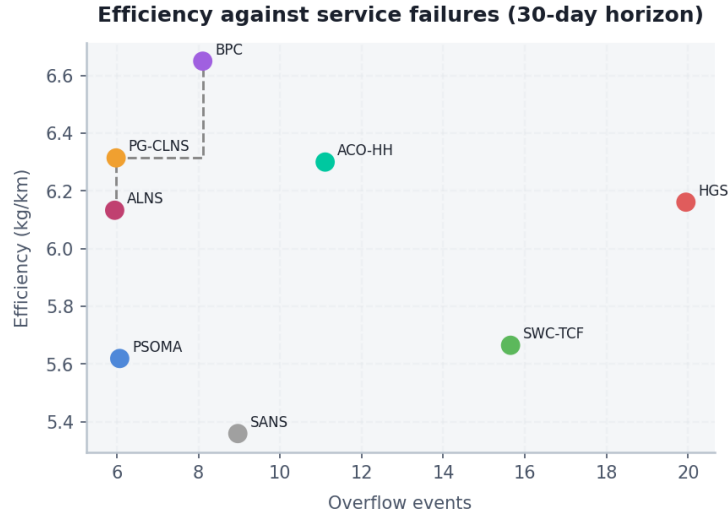
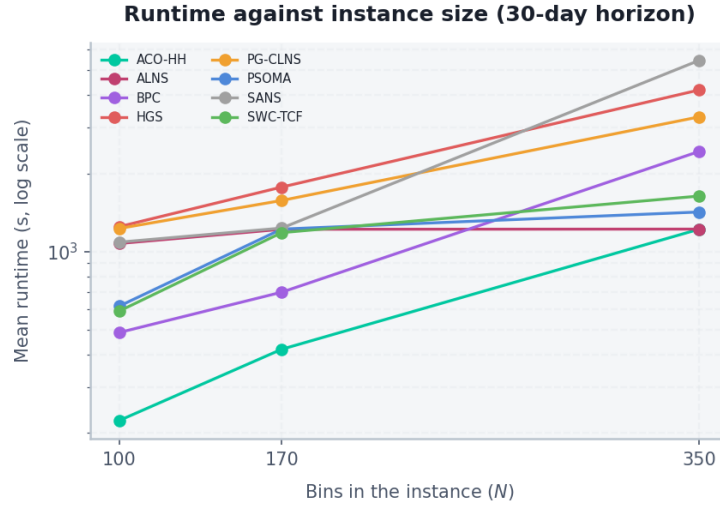


Fig. 4. Mean efficiency against mean overflow events per constructor at the 30-day horizon, with the non-dominated front. Read alongside the medians in Table 1: the horizontal spread is largely tail behaviour.

Table 1. Route constructor performance at the 30-day horizon over the balanced factorial grid (456 runs). Best value per column in bold.

Route constructor	n	kg/km	med.	Ovf.	med.	km	Time (s)
BPC	57	6.65	6.31	8.1	4.0	4,018	1,140
PG-CLNS	57	6.31	6.22	6.0	4.0	4,340	1,967
ACO-HH	57	6.30	6.04	11.1	4.0	4,266	589
HGS	57	6.16	5.99	19.9	6.0	4,553	2,309
ALNS	57	6.13	6.22	5.9	4.0	4,497	1,168
SWC-TCF	57	5.66	5.05	15.6	4.0	5,035	1,109
PSOMA	57	5.62	5.14	6.1	4.0	5,294	1,067
SANS	57	5.36	5.25	9.0	4.0	5,211	2,439

Means and medians over the scenarios that survive the integrity filter. Every constructor faced the identical demand realisation within each scenario and contributes the same 57 runs: the scenario cells containing the degenerate runs of Table 5 were dropped for all constructors, not only for the one that failed them, so no constructor is averaged over a subset that flatters it. Medians are reported alongside means because a single extreme scenario can move a mean substantially at this sample size.

**Fig. 5.** Mean runtime against network size. The exact BPC solver is mid-pack; three meta-heuristics cost more per run than it does.

Mandatory Selection Strategies The selection stage produces a far larger effect than the constructor stage, and it is the paper’s central operational result (Table 2, Fig. 6). Mean efficiency ranges 94% across the five selection variants, from 7.38 kg/km at LM-CF90 down to 3.81 at SL-SL2, against a 24% range across all eight constructors. Which bins are visited matters substantially more than how well the visit is routed — which is an argument for the modular pipeline itself, since a framework that fixed the selection rule and benchmarked only constructors would be optimising the smaller of the two effects.

The five variants order monotonically on every measure simultaneously. Efficiency falls (7.38, 6.37, 5.82, 5.23, 3.81 kg/km), overflow events fall (14.9, 5.5, 4.3, 3.3, 1.2), distance rises (2,940 to 5,711 km) and runtime rises (892 to 1,941 s). The selection threshold therefore behaves as a single dial: collecting later means hauling more per kilometre on shorter routes and overflowing more, and every intermediate setting sits on the curve between those extremes rather than off it. No variant dominates another, and the choice reduces to how an operator prices an overflow against a kilometre.

The three mechanisms are nonetheless not interchangeable at equal service. Look-Ahead attains 5.5 mean overflows at 3,396 km, while Service-Level (SL1) needs 4,147 km to reach 3.3 — so the marginal cost of the last two overflow events is roughly 750 km, an order of magnitude worse than the 456 km that buys the nine events between LM-CF90 and Look-Ahead. Diminishing returns set in sharply, and an operator with a fixed distance budget should read the top of the table rather than the bottom.

One caveat governs this table specifically. The degenerate runs of Sect. 5.3 fall in Look-Ahead and Service-Level (SL2) cells, so removing them removed the study’s hardest scenario from those two variants and not from the other three. Comparing the raw marginals would have credited Look-Ahead with a service level it did not achieve on equal terms. The table is therefore restricted to the slices in which all five variants ran, 80 runs each. This correction is not cosmetic: on the unrestricted data Look-Ahead appears to match LM-CF70 on overflows, and on the balanced data it is clearly worse.

Route Improvers Because the two improvers were run across otherwise identical configurations, they can be compared pairwise rather than marginally: each of the 224 pairs in Table 3 is the same constructor, strategy, network and demand realisation, differing only in improver.

In aggregate CLS wins, and comfortably: 202 of the 224 pairs, mean +0.74 kg/km, on routes 304 km shorter (4,358 against 4,662), at indistinguishable runtime. The mechanism is straightforward — CLS can move a bin between vehicles, and Fast-TSP cannot, so CLS optimises the plan where Fast-TSP only optimises each route’s internal order.

The aggregate, however, conceals the more useful finding. Twenty-one of the 22 CLS losses occur at $N = 350$, and they belong to exactly three constructors: PSOMA (8, mean -2.42), SWC-TCF (8, -2.02) and HGS (6, -1.46). The losses are also substantially deeper than the wins, reaching -3.3 kg/km against a typical

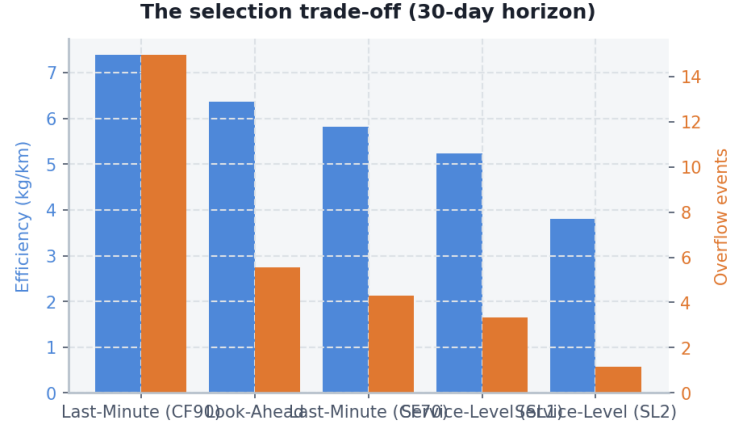


Fig. 6. Efficiency and overflow events by selection strategy, ordered by efficiency, over the slices all five variants share. Both decline monotonically: the threshold is a single dial.

Table 2. Mandatory-selection strategies at the 30-day horizon, ordered by observed collection efficiency, over the slices in which all five variants were run. Efficiency, overflow risk, distance and runtime all order monotonically: the selection threshold is a single dial trading collection efficiency against service reliability.

Selection strategy	<i>n</i>	kg/km	med.	Ovf.	med.	km	Time (s)
Last-Minute (CF90)	80	7.38	7.40	14.9	12.0	2,940	892
Look-Ahead	80	6.37	6.43	5.5	5.0	3,396	914
Last-Minute (CF70)	80	5.82	5.97	4.3	4.0	3,788	1,171
Service-Level (SL1)	80	5.23	5.17	3.3	3.0	4,147	1,376
Service-Level (SL2)	80	3.81	4.08	1.2	1.0	5,711	1,941

CF70/CF90 are Last-Minute critical-fill thresholds of 70% and 90%; SL1/SL2 are Service-Level projection horizons of one and two days. Restricted to the slices shared by all five variants so that the marginal means are comparable: the excluded runs of Table 5 removed the study’s hardest scenario cell from Look-Ahead and Service-Level (SL2) but not from the other three variants, which would otherwise have flattered them. Medians are reported alongside means.

win below +1 (Fig. 7). The correct claim is therefore conditional: *CLS dominates Fast-TSP except on the largest instances for those constructors that produce long routes, where global per-route reordering beats incremental descent*. This is consistent with the two designs — CLS performs first-improvement descent whose cost grows with route length, while Fast-TSP solves each route’s ordering in one shot. An operator choosing an improver should choose by instance size, not by the aggregate.

Overflow counts tie on 181 of the 224 pairs, confirming directly that neither improver alters which bins were selected — exactly as the pipeline’s stage separation intends.

CLS – Fast-TSP efficiency, per matched pair (30d; CLS wins 202 of 224)

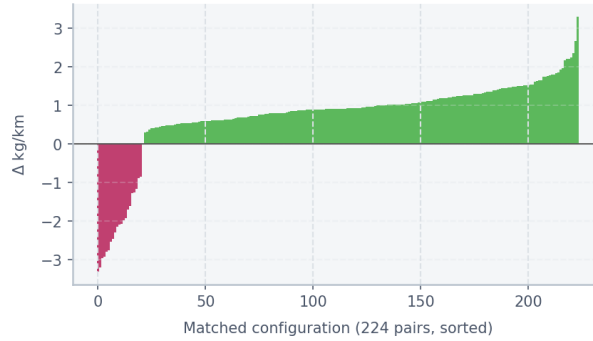


Fig. 7. Per-pair efficiency difference, CLS minus Fast-TSP, sorted. The losses are fewer but deeper than the wins, and all but one occur at $N = 350$.

Table 3. Classical Local Search against Fast-TSP at the 30-day horizon, paired over the 224 configurations that differ only in route improver.

Metric	CLS	Fast-TSP	Δ	CLS wins	Ties
Efficiency (kg/km)	6.39	5.65	0.74	202 / 224	0
Distance travelled (km)	4,358	4,662	-304	202 / 224	0
Overflow events	10.2	10.4	-0.2	20 / 43	181
Runtime (s)	1,412	1,451	-38	185 / 224	0

Pairing rather than averaging: each pair is the same constructor, strategy, region and demand realisation, differing only in improver. Win counts exclude ties, which are reported separately; overflow counts tie often because neither improver changes which bins were selected for collection.

Effect of the Planning Horizon Table 4 compares the 165 configurations observed at both horizons against themselves. Efficiency is essentially horizon-invariant and slightly improving: the mean paired change is $+0.26$ kg/km, positive in 140 of 165 pairs, with HGS the only constructor whose efficiency falls (-0.47). This supports the interpretation that efficiency is governed by route geometry, which a longer horizon does not change.

Overflow counts grow, and the magnitude is informative: the median 90-day to 30-day ratio lies between 2.4 and 3.3 for six of the seven constructors, against a horizon ratio of exactly 3. Overflow events therefore accumulate at an approximately constant *rate* rather than compounding — a longer horizon produces proportionally more failures, not disproportionately more. SWC-TCF is the exception at 5.6, degrading faster than linearly, consistent with its scaling exposure elsewhere in these results.

Two cautions apply. First, this comparison is paired precisely because a cross-constructor comparison at 90 days is not available to us. Second, even the paired comparison is not fully unbiased: a configuration reached the 90-day stage because its 30-day result put it on the Pareto front, so the 30-day arm of each pair is conditioned on having been high, and regression to the mean works against the measured change. The direction of these results should be trusted; their magnitude should be read as conservative.

Table 4. Horizon sensitivity, computed on the 165 policy configurations observed at both horizons and compared against themselves.

Route constructor	Pairs	kg/km 30d	kg/km 90d	Ovf. 30d	Ovf. 90d
ACO-HH	28	5.35	5.68	3.2	11.5
BPC	57	6.65	6.96	8.1	28.5
HGS	12	7.75	7.28	38.1	105.9
PG-CLNS	40	6.26	6.56	5.0	16.3
PSOMA	11	4.49	4.79	2.3	7.3
SANS	12	4.66	4.93	2.8	8.7
SWC-TCF	5	5.83	6.33	4.4	25.6

Only configurations on the 30-day Pareto front were re-run at 90 days, so the 90-day sample is conditioned on 30-day performance. Constructors are therefore compared against themselves across horizons and never against each other at 90 days.

Scenario Effects The two generating processes produce markedly different regimes and should not be pooled. Gamma-3 yields 7.35 kg/km mean efficiency against 4.83 for the Empirical process, and 15.0 mean overflow events against 5.9 — it is simply a heavier load, generating more waste per bin-day, which makes collection more efficient per kilometre and service failure more likely at the same time. Any conclusion stated across both processes without qualification would be misleading, and we have stated results per process wherever the two disagree.

Network size does not act monotonically on efficiency (5.97, 5.26 and 6.99 kg/km at $N = 100, 170$ and 350), because the two networks differ in bin density as well as in size and Figueira da Foz is the denser. It does act monotonically on cost: mean distance rises from 2,343 to 7,618 km and mean runtime from 819 to 2,608 s. Overflow events rise sharply at $N = 350$ (21.6 against 4.9 and 5.9), which is where every constructor’s tail behaviour — including the three degenerate runs of Sect. 5.3 — is concentrated.

Table 5. Runs excluded from all aggregate statistics as degenerate rather than merely poor.

Constructor	H	Scenario	Policy	kg	Short.	Ovf.
SWC-TCF	90	FF 350	Gamma-3/LA/FTSP	31,164	86%	23,886
SWC-TCF	30	FF 350	Gamma-3/LA/FTSP	37,065	47%	2,166
SWC-TCF	30	FF 350	Gamma-3/LA/CLS	37,065	47%	2,168
SWC-TCF	30	FF 350	Gamma-3/SLSL2/FTSP	51,086	30%	757

H is the horizon in days and “FF 350” the Figueira da Foz network at $N = 350$. Within a scenario every constructor collects from the same waste realisation, so collected tonnage is comparable by construction. These runs fell more than 20% below their scenario median and did not complete; their overflow counts measure truncation, not policy behaviour.

6 Conclusion and Future Work

Should write some text here after finishing up the Experimental Evaluation.

Acknowledgments. This study was funded by the WSmart Route+ project (grant number BI|2024/549), and carried out at INESC-ID.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Archetti, C., Speranza, M.G., Vigo, D.: Chapter 10: Vehicle Routing Problems with Profits, pp. 273–297. <https://doi.org/10.1137/1.9781611973594.ch10>, <https://epubs.siam.org/doi/abs/10.1137/1.9781611973594.ch10>
2. Hess, C., Dragomir, A.G., Doerner, K.F., Vigo, D.: Waste collection routing: a survey on problems and methods. *Central European Journal of Operations Research* **32**(2), 399–434 (2024)
3. Jorge, D., Antunes, A.P., Ramos, T.R.P., Barbosa-Póvoa, A.P.: A hybrid meta-heuristic for smart waste collection problems with workload concerns. *Computers & Operations Research* **137**, 105518 (2022)

4. Joshi, C.K., Cappart, Q., Rousseau, L.M., Laurent, T.: Learning the travelling salesperson problem requires rethinking generalization. arXiv preprint arXiv:2006.07054 (2020)
5. Joshi, C.K., Laurent, T., Bresson, X.: An efficient graph convolutional network technique for the travelling salesman problem. arXiv preprint arXiv:1906.01227 (2019)
6. Karalias, N., Loukas, A.: Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems* **33**, 6659–6672 (2020)
7. Kool, W., van Hoof, H., Welling, M.: Attention, learn to solve routing problems! In: *International Conference on Learning Representations* (2018), <https://api.semanticscholar.org/CorpusID:59608816>
8. Kucharska, E.: Dynamic vehicle routing problem—predictive and unexpected customer availability. *Symmetry* **11**(4) (2019). <https://doi.org/10.3390/sym11040546>, <https://www.mdpi.com/2073-8994/11/4/546>
9. Kwon, Y.D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S.: Pomo: Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems* **33**, 21188–21198 (2020)
10. Liu, F., Xialiang, T., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., Zhang, Q.: Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In: *Forty-first International Conference on Machine Learning* (2024)
11. Lopes, M., Ramos, T.R.: Efficient sensor placement and online scheduling of bin collection. *Computers & Operations Research* **151**, 106113 (2023)
12. Ma, Y., Cao, Z., Chee, Y.M.: Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt. *Advances in Neural Information Processing Systems* **36** (2024)
13. de Morais, C.S., Ramos, T.R.P., Lopes, M., Barbosa-Póvoa, A.P.: A data-driven optimization approach to plan smart waste collection operations. *International Transactions in Operational Research* **31**(4), 2178–2208 (2024)
14. Ramos, T.R.P., de Morais, C.S., Barbosa-Póvoa, A.P.: The smart waste collection routing problem: Alternative operational management approaches. *Expert Systems with Applications* **103**, 146–158 (2018). <https://doi.org/https://doi.org/10.1016/j.eswa.2018.03.001>, <https://www.sciencedirect.com/science/article/pii/S0957417418301404>
15. Sun, Z., Yang, Y.: Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in Neural Information Processing Systems* **36**, 3706–3731 (2023)
16. Vidal, T.: Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood. *Computers & Operations Research* **140**, 105643 (2022). <https://doi.org/https://doi.org/10.1016/j.cor.2021.105643>, <https://www.sciencedirect.com/science/article/pii/S030505482100349X>
17. Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. *Advances in neural information processing systems* **28** (2015)
18. Wu, X., Wang, D., Wen, L., Xiao, Y., Wu, C., Wu, Y., Yu, C., Maskell, D.L., Zhou, Y.: Neural combinatorial optimization algorithms for solving vehicle routing problems: A comprehensive survey with perspectives. arXiv preprint arXiv:2406.00415 (2024)
19. Zhang, Z., Che, O., Cheang, B., Lim, A., Qin, H.: A memetic algorithm for the multiperiod vehicle routing problem with profit. *Eur. J. Oper. Res.* **229**(3), 573–584 (2013). <https://doi.org/10.1016/J.EJOR.2012.11.059>, <https://doi.org/10.1016/j.ejor.2012.11.059>